

SlotMate – Smart University Slot Exchange and Matching System

Abstract

SlotMate is a web-based platform that is designed to solve the problem of inefficient course slot exchange among university students. In many universities, students are often assigned class sections, faculty members or time slots that do not match their preferences. The traditional method of finding a compatible exchange partner through social media groups or word-of-mouth is time-consuming and unreliable.

SlotMate provides a centralized platform where students can post their current slot details along with their preferred slot requirements. An automated matching algorithm then identifies and ranks compatible exchange partners based on a mutual compatibility score. The system was built using Django and Python for the backend, HTML, CSS, and JavaScript for the frontend, and SQLite as the database.

This report documents the full development process of SlotMate including problem identification, requirement analysis, system design, database structure, implementation, testing, and future recommendations.

Keywords: Slot Exchange, Django, Web Application, Matching System, University Scheduling.

Chapter 1: Introduction

1.1 Background

University students frequently face scheduling conflicts after being assigned class sections or faculty members that conflict with their academic, personal, or professional commitments. The process of resolving these conflicts has traditionally relied on informal methods such as Facebook groups, WhatsApp chats, and notice boards. These approaches are unstructured, inefficient, and often result in students failing to find suitable exchange partners despite compatible matches existing within the same institution.

SlotMate was developed to address this gap by providing a dedicated, intelligent platform that automates the process of finding and ranking compatible slot exchange partners.

1.2 Problem Statement

The following problems motivated the development of SlotMate:

- i Students have no centralized platform to post and search slot exchange requests.
- ii Finding a mutually compatible exchange partner manually is time-consuming.
- iii Informal communication channels do not support structured matching criteria.
- iv Compatible exchange pairs are frequently missed due to limited reach.
- v There is no system to rank or score the quality of a potential exchange match.

1.3 Objectives

General objective: To develop a web-based system that automates and simplifies the process of slot exchange among university students.

Specific Objectives:

- i Allow students to create and manage slot exchange requests.
- ii Securely store all exchange request data in a relational database.
- iii Automatically generate matches between students with compatible slots.
- iv Calculate and display compatibility scores for each match.
- v Present ranked match results to help students make informed decisions.
- vi Provide an intuitive dashboard with an overview of requests and matches.

1.4 Scope

The project includes:

1. User registration and login with secure authentication
2. Student profile creation and management
3. Slot exchange request creation, viewing, and deletion
4. Automated match generation with compatibility scoring
5. Dashboard with statistics and active request overview.

The project does not include:

1. Integration with official university registration systems
2. Administrative approval or moderation workflows
3. Real-time messaging or in-app chat between students

4. No payment system.

Chapter 2: Literature Review

Traditional methods of slot exchange rely heavily on social media platforms and informal communication. These methods lack structure, have limited reach, and provide no mechanism for evaluating the quality of a potential match. Students must manually compare their own slot details with those of others, which is inefficient and error-prone. These platforms define user requirements, evaluate candidate profiles against those requirements, and rank results by a compatibility metric.

SlotMate applies this principle to the academic scheduling domain. By defining structured slot attributes such as course code, section, faculty and time and evaluating preferences bidirectionally, the system can produce ranked match results that far exceed what manual searching can achieve. The use of a mutual compatibility score ensures that both parties in a potential exchange are well-served by the outcome, not just one.

Chapter 3: Requirement Analysis

3.1 Functional Requirements

User Registration

- i Users can create accounts by providing their name, email, password, university, and department.
- ii Registration data is validated and securely stored in the database.

User Authentication

- i Users can log in using their registered credentials.
- ii Sessions are managed securely and unauthorized users are redirected to the login page.

Slot Request Management

- i Students can create exchange requests to specify their current slot and preferred slot.
- ii Students can view all their active requests.
- iii Students can delete requests they no longer need.

Matching System

- i The system automatically generates matches when requests are evaluated.

- ii Compatibility scores are calculated for each match pair from both directions.
- iii Matches are displayed in descending order of mutual compatibility score.

Dashboard

- i Displays the number of active requests and total matches found.
- ii Provides an overview of recent match activity.

3.2 Non-Functional Requirements

Performance:

- i Fast request processing.
- ii Efficient database queries.

Security:

- i Password protection
- ii User authentication
- iii Access control.

Usability

- i Simple user interface
- ii Responsive design

Reliability

- i Stable operation
- ii Data consistency.

Chapter 4: System Design

4.1 System Architecture

SlotMate is built on the Model-View-Template (MVT) architectural pattern provided by the Django framework.

Model: Defines the data structure and manages all database interactions through Django's ORM.

View: Contains the business logic that processes HTTP requests, interacts with models, and returns HTTP responses.

Template: Defines the HTML structure of each page, rendered dynamically with data passed from the views.

Core Components:

Models:

- i StudentProfile,
- ii SlotRequest,
- iii Match

Views:

- i Register,
- ii Login,
- iii Dashboard,
- iv Request Management,
- v Match Management.

Templates:

- i index.html
- ii login.html
- iii register.html
- iv dashboard.html
- v my-requests.html
- vi matches.html
- vii match-details.html

4.2 Database design

The system uses three primary database models.

StudentProfile

Field	Type
User	OneToOneField
University	CharField
Department	TextField

SlotRequest

Field	Type
-------	------

current-course-code	CharField
current-section	CharField
current-faculty	CharField
current-time	CharField
preferred-course-code	CharField
preferred-faculty	CharField
preferred-time	CharField
any-day	BooleanField
any-time	CharField

Match

Field	Type
user_a	ForeignKey
user_b	ForeignKey
request_a	ForeignKey
request_b	ForeignKey
score_a_to_b	FloatField
score_b_to_a	FloatField
mutual_score	FloatField

Chapter 5: Implementation

5.1 Technologies Used

Backend

- i Python 3
- ii Django

Frontend

- i HTML5
- ii CSS3
- iii JavaScript

Database

- i SQLite

Development Tools

- ii Visual Studio Code
- iii GitHub
- iv Chrome Browser

5.2 key Modules

models.py- Defines all database models: StudentProfile, SlotRequest, Match, RevealRequest, and Notification.

views.py - Handles all request/response logic for every page and action in the application.

matching.py- Core matching algorithm that evaluates all active requests pairwise, scores them, and returns ranked results.

urls.py - Maps all URL paths to their corresponding view functions.

templates/ - HTML files for all application pages.

5.3 Matching Algorithm

The matching algorithm compares

- i Course preferences
- ii Section preferences
- iii Faculty preferences
- iv Time preferences

For each pair of requests

1. Compatibility scores are calculated
2. Mutual compatibility
3. Requests exceeding the threshold are stored as matches
4. Matches are ranked by mutual score

Chapter 6: Testing and Results

6.1 Test Cases Registration Test

Expected Result: user account creates successfully

Actual Result: Passed

Login Test

Expected Result: Authentication user enters dashboard

Actual Result: Passed

Create Request Test

Expected Result: Request saved successfully

Actual Result: Passed

Match Generation Test

Expected Result: Compatible users appear in matches

Actual Result: Passed

Delete Request Test

Expected Result: Request removed successfully

Actual Result: Passed

6.2 Results

All test cases passed successfully. The system correctly handled user registration and authentication, persisted slot requests, generated ranked matches based on compatibility scoring, enforced access control, and prevented duplicate match entries. The matching algorithm produced accurate scores that directly reflected the degree of preference alignment between each pair.

Chapter 7: Challenges Faced

Matching Algorithm Design: Deciding how to weight each criterion and setting the right compatibility threshold took several rounds of testing and adjustment.

Duplicate Match Prevention: Ensuring each pair appears only once regardless of which user triggered the evaluation required a careful deduplication step.

Reveal Request Logic: Managing the three-state status system (pending, accepted, rejected) and ensuring both users are notified correctly required careful view and model design.

Notification Triggers: Identifying all the events that should generate notifications and ensuring they fire correctly without duplication required thorough testing.

Access Control: Consistently protecting all authenticated pages across every view function required disciplined use of Django's login required decorator.

Chapter 8: Future Enhancements

In-App Chat: Add direct messaging between matched students so they can coordinate the exchange without leaving the platform.

University System Integration: Connect to official university APIs to automatically verify slot data and reduce manual input errors.

Mobile Application: Develop iOS and Android apps for broader accessibility.

Machine Learning Matching: Use historical exchange data to train a model that improves match quality beyond the current rule-based scoring system.

Admin Panel: Add an administrative interface for university staff to monitor exchange activity and manage users.

Chapter 9: Conclusion

SlotMate successfully automates the process of finding compatible slot exchange partners for university students. The system covers the full exchange workflow from posting a request, to finding a match, to sending a reveal request, to receiving contact details all

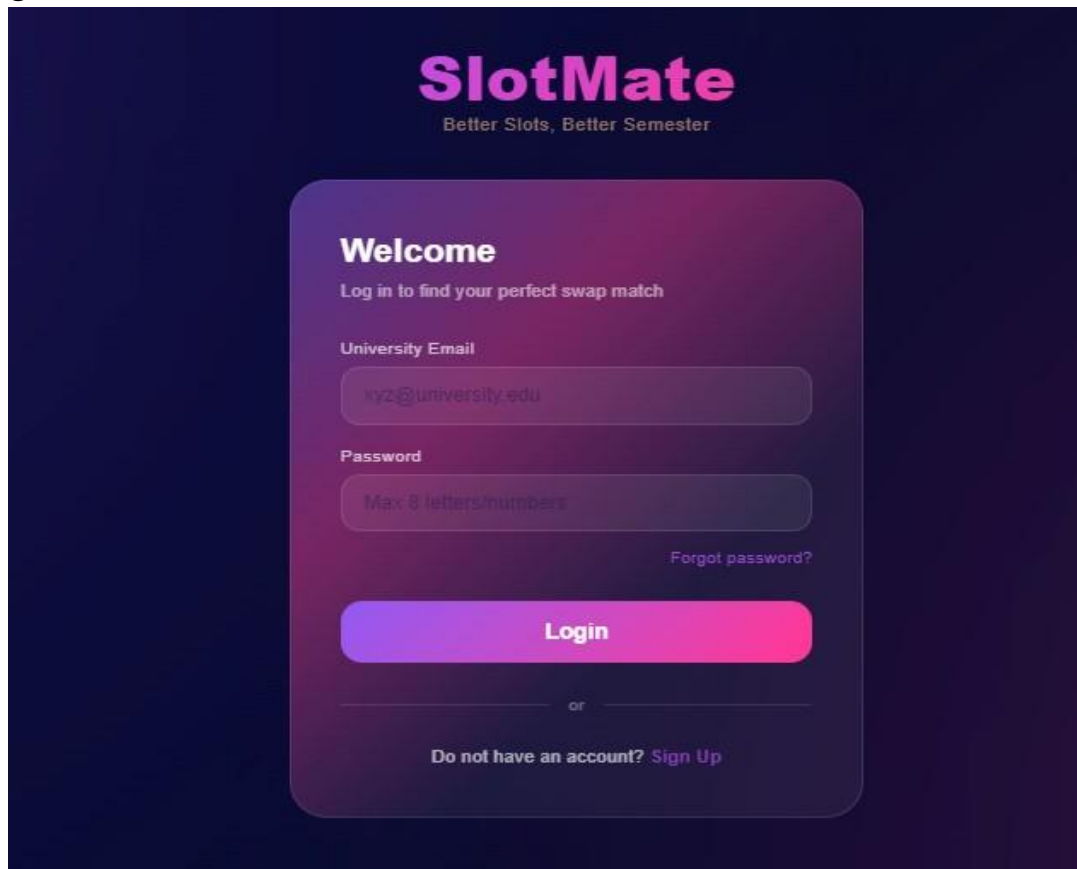
within a single platform. The project demonstrates practical skills in Django, database design, algorithmic thinking, and frontend development. All objectives were met and the system performed correctly across all tested scenarios.

References:

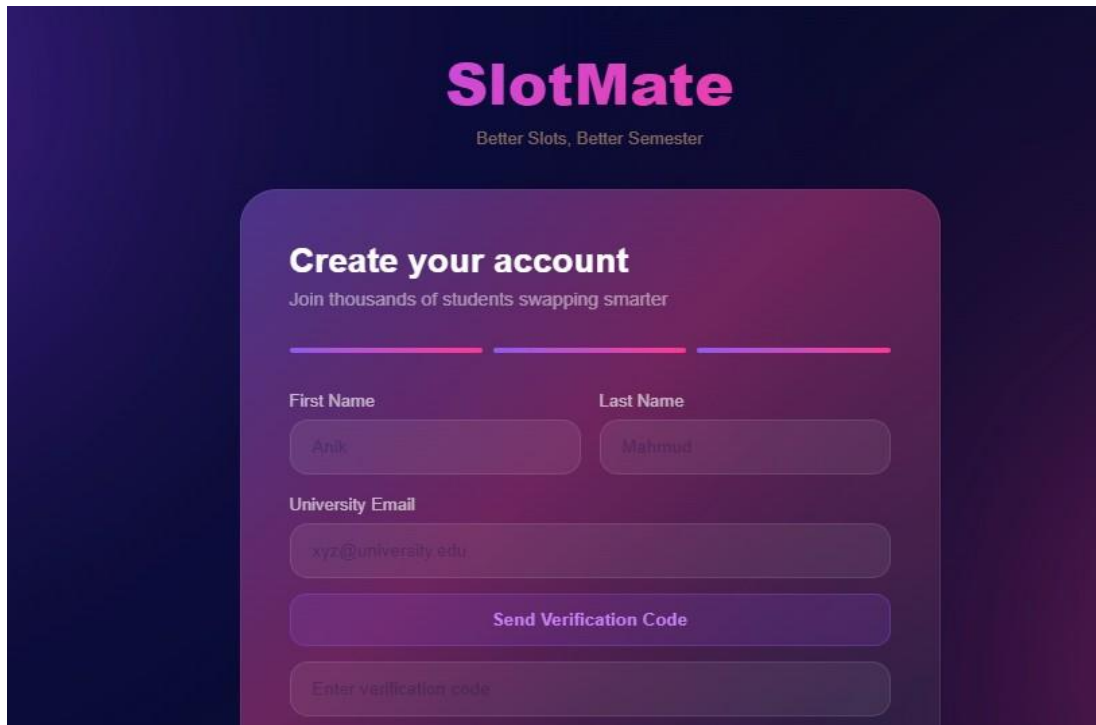
1. Django Software Foundation. (2024). *Django Documentation*. <https://docs.djangoproject.com>
2. Python Software Foundation. (2024). *Python Documentation*. <https://docs.python.org>
3. Mozilla Developer Network. (2024). *MDN Web Docs*. <https://developer.mozilla.org>
4. W3Schools. (2024). *Web Development Reference*. <https://www.w3schools.com>
5. SQLite Consortium. (2024). *SQLite Documentation*. <https://www.sqlite.org>

Screenshots:

Login page:



Register page:

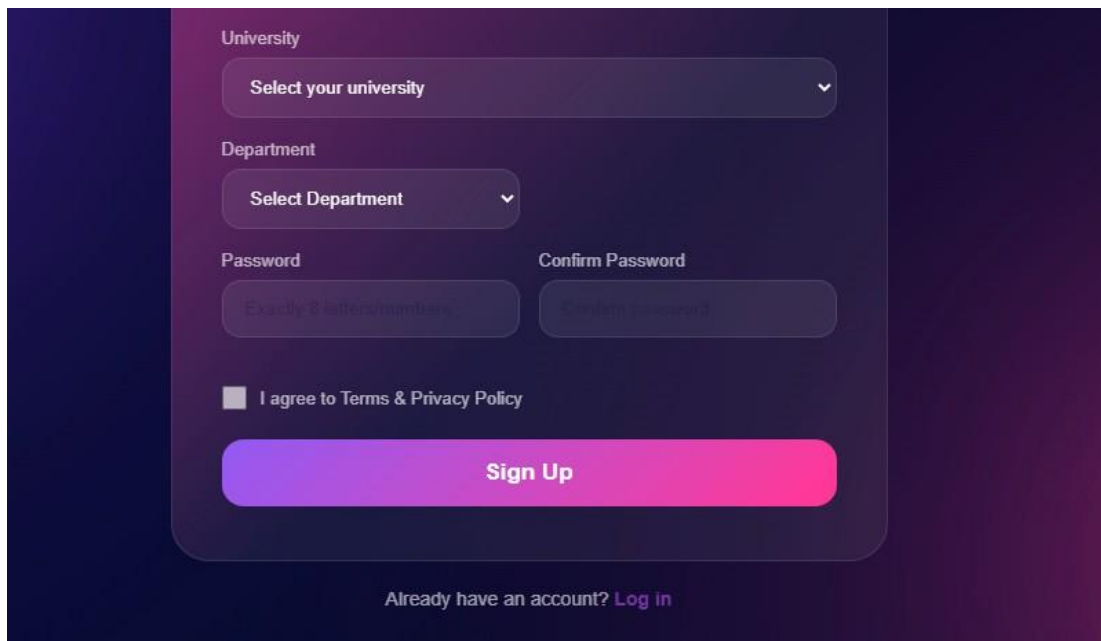


The image shows the 'Create your account' section of the SlotMate registration page. The background is a dark purple gradient. At the top, the 'SlotMate' logo is in a bold, pink font, with the tagline 'Better Slots, Better Semester' in a smaller, white font below it. The registration form is a rounded rectangle with a purple-to-pink gradient. It contains the following elements: a title 'Create your account', a subtext 'Join thousands of students swapping smarter', a horizontal separator line, two input fields for 'First Name' (containing 'Anik') and 'Last Name' (containing 'Mahmud'), an input field for 'University Email' (containing 'xyz@university.edu'), a 'Send Verification Code' button, and an input field for 'Enter verification code'.

SlotMate
Better Slots, Better Semester

Create your account
Join thousands of students swapping smarter

First Name: Anik
Last Name: Mahmud
University Email: xyz@university.edu
Send Verification Code
Enter verification code



The image shows the 'Sign Up' section of the SlotMate registration page. It features a rounded rectangle with a purple-to-pink gradient. The form includes: a 'University' dropdown menu with the text 'Select your university', a 'Department' dropdown menu with the text 'Select Department', two input fields for 'Password' (containing 'Exactly 8 letters/numbers') and 'Confirm Password' (containing 'Confirm password'), a checkbox labeled 'I agree to Terms & Privacy Policy', a large 'Sign Up' button, and a link 'Log in' at the bottom with the text 'Already have an account?'.

University: Select your university
Department: Select Department
Password: Exactly 8 letters/numbers
Confirm Password: Confirm password
☐ I agree to Terms & Privacy Policy
Sign Up
Already have an account? [Log in](#)

DashBoard page:

SlotMate

Better Slots, Better Semester

Home

Create Request

Matches

Notifications1

Profile▼

Hello, Nusrat Rafi

Southeast University

CSE

100%

CSE461 ↔ CSE341

Section: 1 ↔ 1

Faculty: RFD ↔ MAR

Time: 11:30 AM ↔ 10:00 AM

View Details

93%

CSE381 ↔ CSE456

Section: 1 ↔ 2

Faculty: SBB ↔ NON

Time: 11:30 AM ↔ 10:00 AM

View Details

65%

cse161 ↔ CSE181

Section: 1 ↔ 2

Faculty: ANMS ↔ NON

Time: 10:00 AM ↔ 1:30 PM

View Details

Live Match Feed

SlotMate

Better Slots, Better Semester

Home

Create Request

Matches

Notifications1

Profile▼

Your Swap Snapshot

Active Requests

3

View All

Total Matches

3

View All

Successful

2

View All

"The right match can change your whole semester" ✨

Graphical Match Insight

See how your preferences are matching!

Match Strength

Course

Day

Time

Faculty

Section

View Top Match Details

Quick Tip: Be specific with time preferences to get better matches!

Create -Request Page:

SlotMate

Better Slots, Better Semester

Back To Home

Create Swap Request

Current Course Info

Course Code (e.g. CSE472)

Section (e.g. 1)

Faculty Initial (e.g. ANMS)

Current Day

Saturday

Sunday

Monday

Tuesday

Wednesday

Thursday

Current Time

8:00 AM

8:30 AM

10:00 AM

11:30 AM

1:30 PM

3:00 PM

4:30 AM

SlotMate

Better Slots, Better Semester

Back To Home

Preferred Course Info

Preferred Course (CSE361)

Preferred Section (e.g. 2)

Preferred Faculty Initial (e.g. NON)

Preferred Days

Saturday

Sunday

Monday

Tuesday

Wednesday

Thursday

Preferred Time

8:00 AM

8:30 AM

10:00 AM

11:30 AM

1:30 PM

3:00 PM

4:30 AM

☐ Any Time

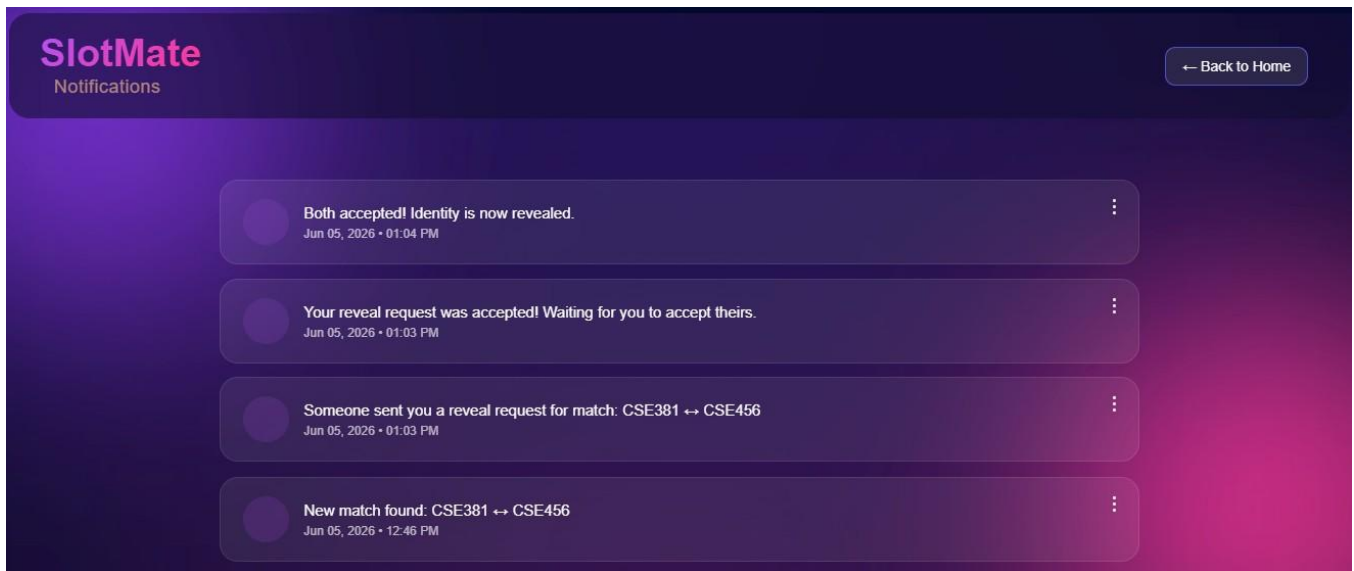
☐ Any Day

☐ Any Faculty

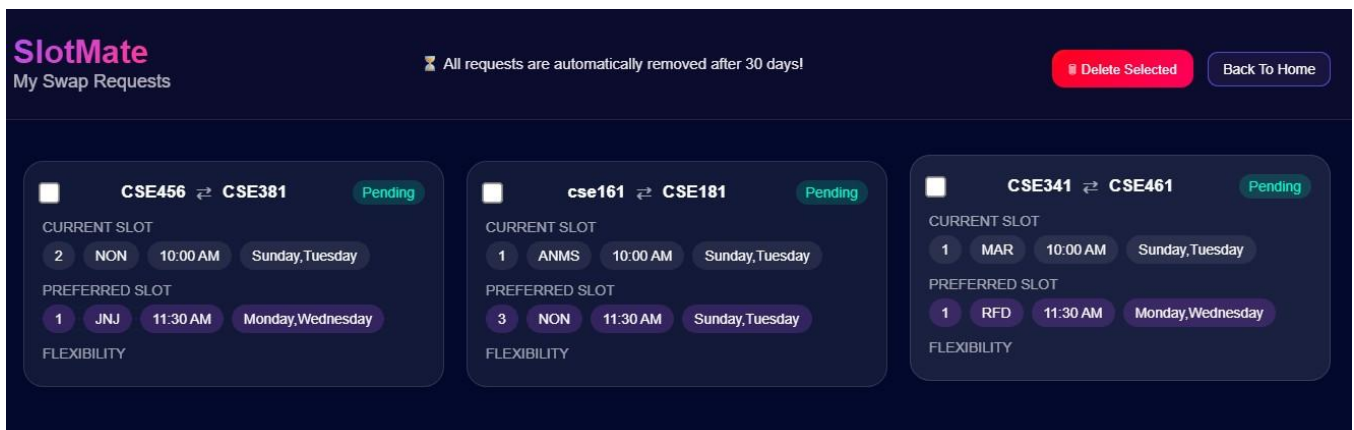
☐ Any Section

Submit Request

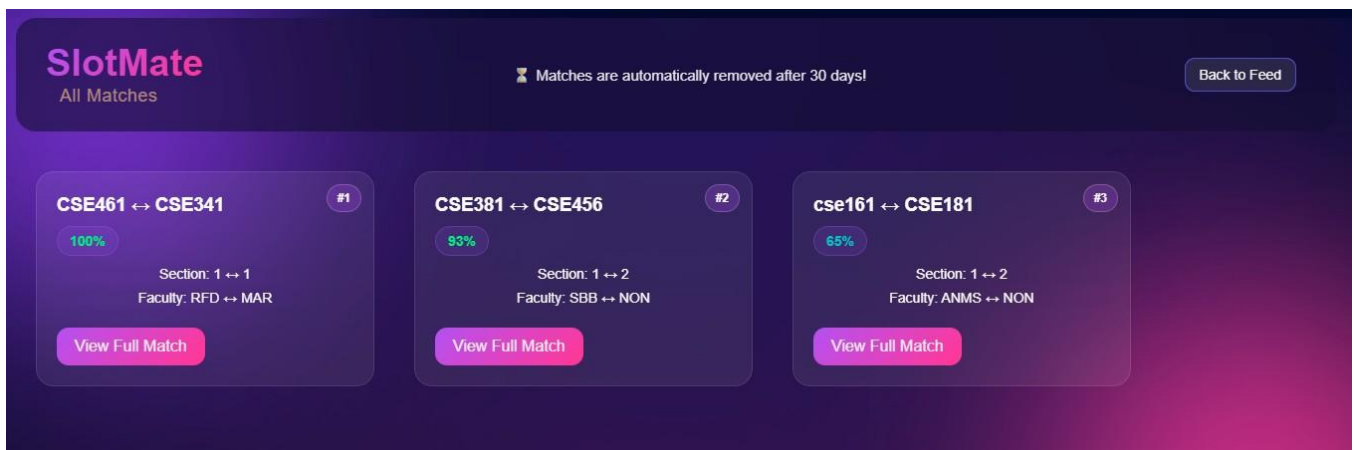
Notification page:



My request page:



Matches Page:



Match-details page:

SlotMate
Match Details

Back To Matches

Compatibility Match Found

93%

CSE381 ↔ CSE456

User A	
Course	CSE456
Section	2
Faculty	NON
Time	10:00 AM
Days	Sunday,Tuesday

User B	
Course	CSE381
Section	1
Faculty	SBB
Time	11:30 AM
Days	Monday,Wednesday

Reveal Identity:

Want to Reveal Identity?

Both users must accept before personal details are shown.

Send Reveal Request

Want to Reveal Identity?

Both users must accept before personal details are shown.

Waiting for the other user to send their request...

Want to Reveal Identity?

Both users must accept before personal details are shown.

Someone wants to reveal identity with you!

Accept

Reject

Identity Revealed
Nusrat Nawal

2023100000274@seu.edu.bd